GraphRAG — Knowledge Graphs for Retrieval-Augmented Generation

> 🎯 **Goal**: Understand why vanilla RAG fails on global/holistic questions and how GraphRAG solves this with knowledge graphs, community detection, and hierarchical summaries. Build a practical iterative book-summarization pipeline.

> **Prerequisites**: Day 5 — RAG + Vector Memory (embeddings, chunking, hybrid search)

Table of Contents

1. Why Vanilla RAG Fails on Global Questions
2. What GraphRAG Is
3. GraphRAG Architecture (Step by Step)
4. Local vs Global Search
5. When to Use GraphRAG vs Vanilla RAG
6. Quality Monitoring & Evaluation
7. Practical Solution: Iterative Book Summarization
    - Approach 1: Single-Pass LLM
    - Approach 2: Chunk-and-Summarize (Map-Reduce)
    - Approach 3: Sliding Window with Running Context
    - Approach 4: Entity-Tracking Map-Reduce
    - Approach 5: GraphRAG + Iterative Refinement ✅
    - The Recommended Solution: GraphRAG 5-Pass Architecture
8. Tools & Frameworks
9. References

Why Vanilla RAG Fails on Global Questions

Vanilla RAG (chunk → embed → retrieve → generate) works great for **local** questions where the answer lives in one or two chunks:

```
✅ LOCAL QUESTION (Vanilla RAG works):
  "What year was the company founded?"
  → One chunk contains "Founded in 2015" → perfect answer
```

```
❌ GLOBAL QUESTION (Vanilla RAG fails):
  "What are the main themes across all company memos this year?"
  → Answer is spread across 500 documents
  → No single chunk contains the full answer
  → Top-k retrieval returns 5-10 chunks out of 500 → misses 98% of the picture
```

The fundamental problem:

```
┌──────────────────────────────────────────────────────────────────────┐
│                    WHY VECTOR SEARCH MISSES THE BIG PICTURE          │
│                                                                      │
│  Imagine a 800-page novel. You ask:                                  │
│  "Summarize the story without missing any characters or key events"  │
│                                                                      │
│  Vanilla RAG:                                                        │
│  1. Chunks the book into 2000 passages                               │
│  2. Embeds your question                                             │
│  3. Retrieves top 10 most similar passages (0.5% of the book!)       │
│  4. LLM summarizes those 10 passages                                 │
│                                                                      │
│  Result: You get a summary of 10 random-ish passages.                │
│  Missing: 99.5% of characters, events, subplots, foreshadowing       │
│                                                                      │
│  The problem isn't retrieval quality — it's that the QUESTION        │
│  requires information from EVERYWHERE, not from a few chunks.        │
└──────────────────────────────────────────────────────────────────────┘
```

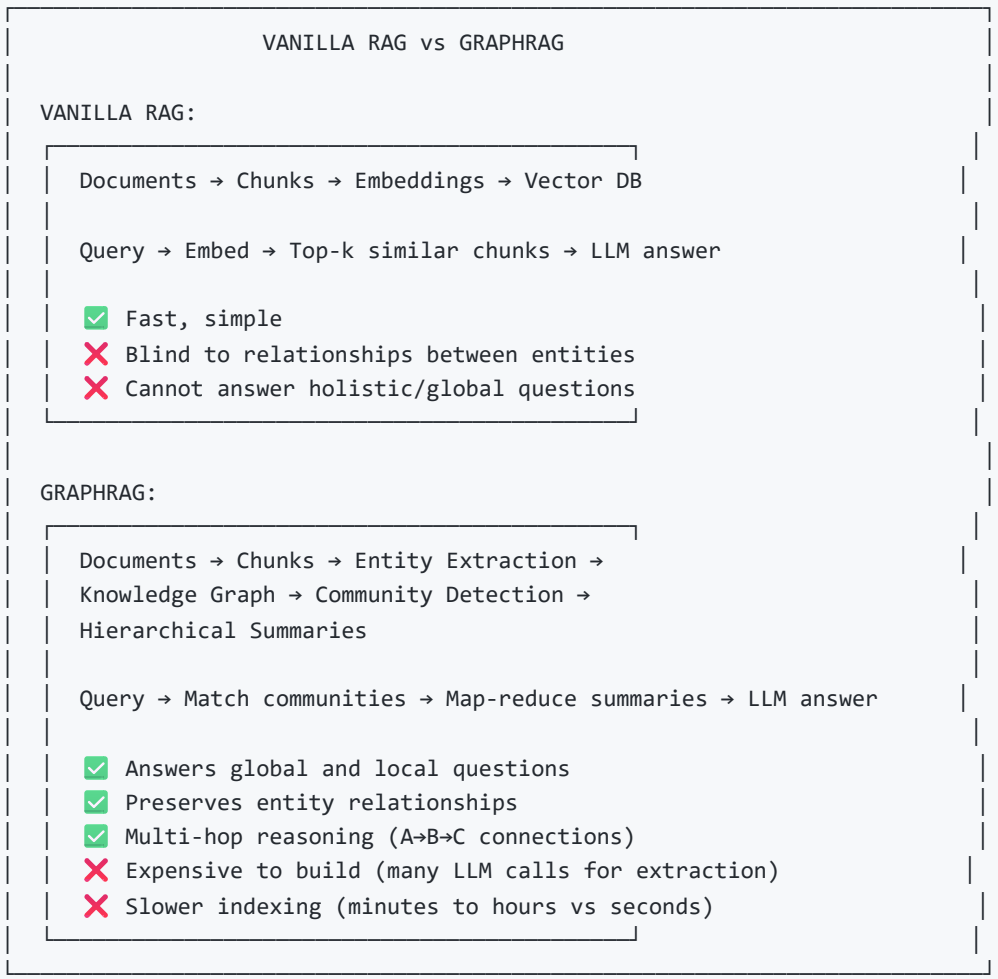
Three failure modes of vanilla RAG on global questions:

Failure Mode	Example	Why It Happens
Information scatter	"What are the themes in this book?"	Answer requires synthesizing ALL chapters, not a few chunks
Relationship blindness	"How are characters A and B connected?"	Their relationship is described across 50 scattered passages
Structural blindness	"How does event X cause event Y 200 pages later?"	Cause and effect are too far apart for any chunk to capture both

---

What GraphRAG Is

GraphRAG = Graph-based Retrieval-Augmented Generation. Instead of treating documents as flat text chunks, it builds a **knowledge graph** of entities and relationships, then uses **community detection** to create hierarchical summaries that capture the big picture.

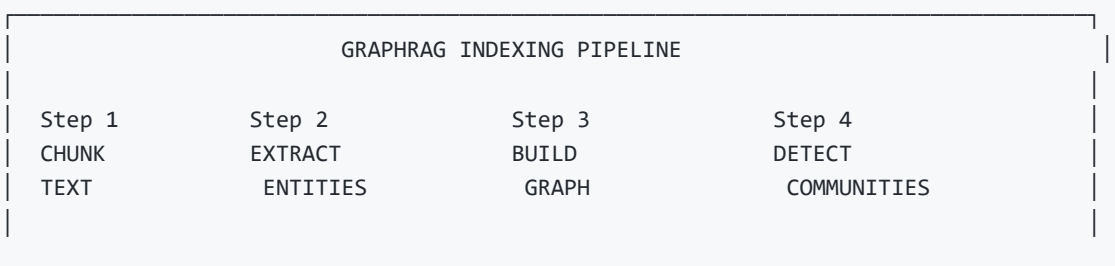


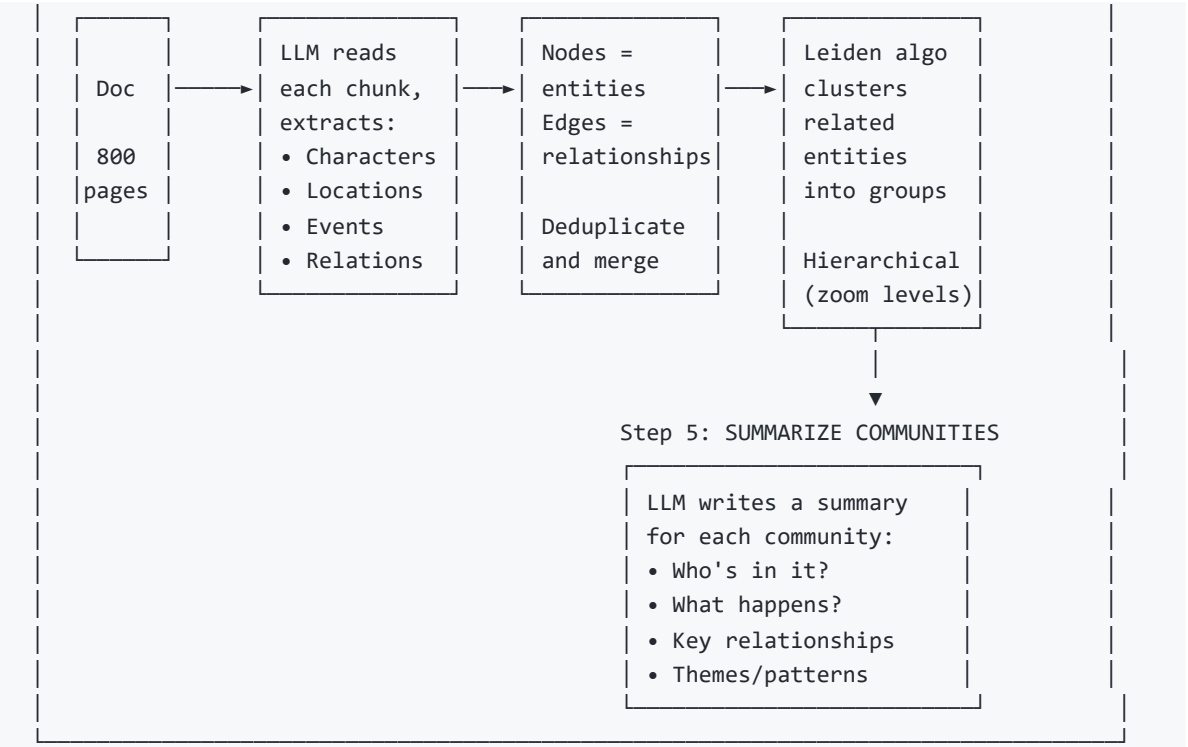
Developed by: Microsoft Research (2024). Paper: "From Local to Global: A Graph RAG Approach to Query-Focused Summarization" (arXiv:2404.16130). Open-sourced at github.com/microsoft/graphrag.

GraphRAG Architecture (Step by Step)

GraphRAG has two phases: **Indexing** (build the graph) and **Querying** (answer questions). Indexing is expensive but done once; querying is fast.

Indexing Pipeline





Let's walk through each step with a concrete example — indexing a novel:

Step 1: Chunk the Text

Same as vanilla RAG. Split the document into overlapping chunks of ~300-600 tokens.

```
Chapter 1, chunk 1: "Harry had been living with the Dursleys for ten years,
                     ten miserable years, ever since he'd been left on their
                     doorstep as a baby..."

Chapter 1, chunk 2: "Hagrid appeared at the door of the hut on the rock,
                     enormous and wild-looking. 'Harry — yer a wizard'..."
```

Step 2: Entity & Relationship Extraction (LLM-Powered)

For each chunk, an LLM extracts **entities** (people, places, objects, events) and **relationships** between them. This is the most expensive step — every chunk requires an LLM call.

```
Prompt to LLM (simplified):

  "Extract all entities and relationships from this text.
   For each entity, give: name, type, description.
   For each relationship, give: source, target, description, strength."

Chunk input:
  "Hagrid arrived and told Harry he was a wizard.
   Harry was shocked — the Dursleys had hidden this from him."
```

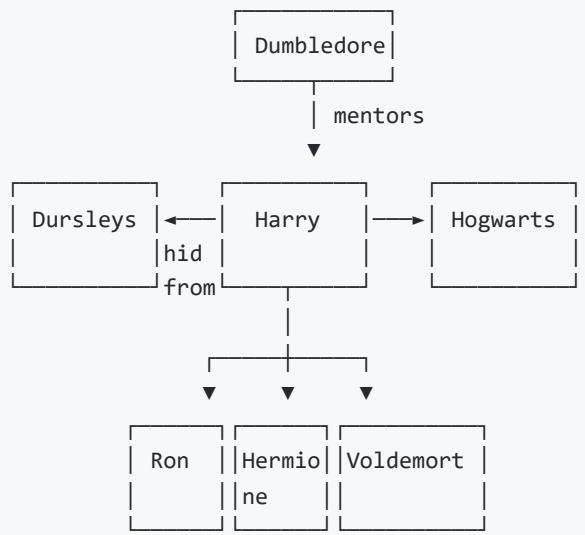
```
LLM output:
  ENTITIES:
    (Harry, person, "An orphan boy living with the Dursleys")
    (Hagrid, person, "A giant man who delivers Harry's Hogwarts letter")
    (Dursleys, family, "Harry's aunt and uncle, non-magical")

  RELATIONSHIPS:
    (Hagrid) ──reveals truth to──► (Harry)         strength: 9
    (Dursleys) ──hid wizardry from──► (Harry)      strength: 8
    (Harry) ──lives with──► (Dursleys)             strength: 7
```

Key detail: Extraction runs on every chunk, so the same entity (e.g., "Harry") appears many times. The next step deduplicates and merges these.

Step 3: Build the Knowledge Graph

Merge all extracted entities and relationships into a single graph. Deduplicate entities (e.g., "Harry", "Harry Potter", "the boy" → one node).

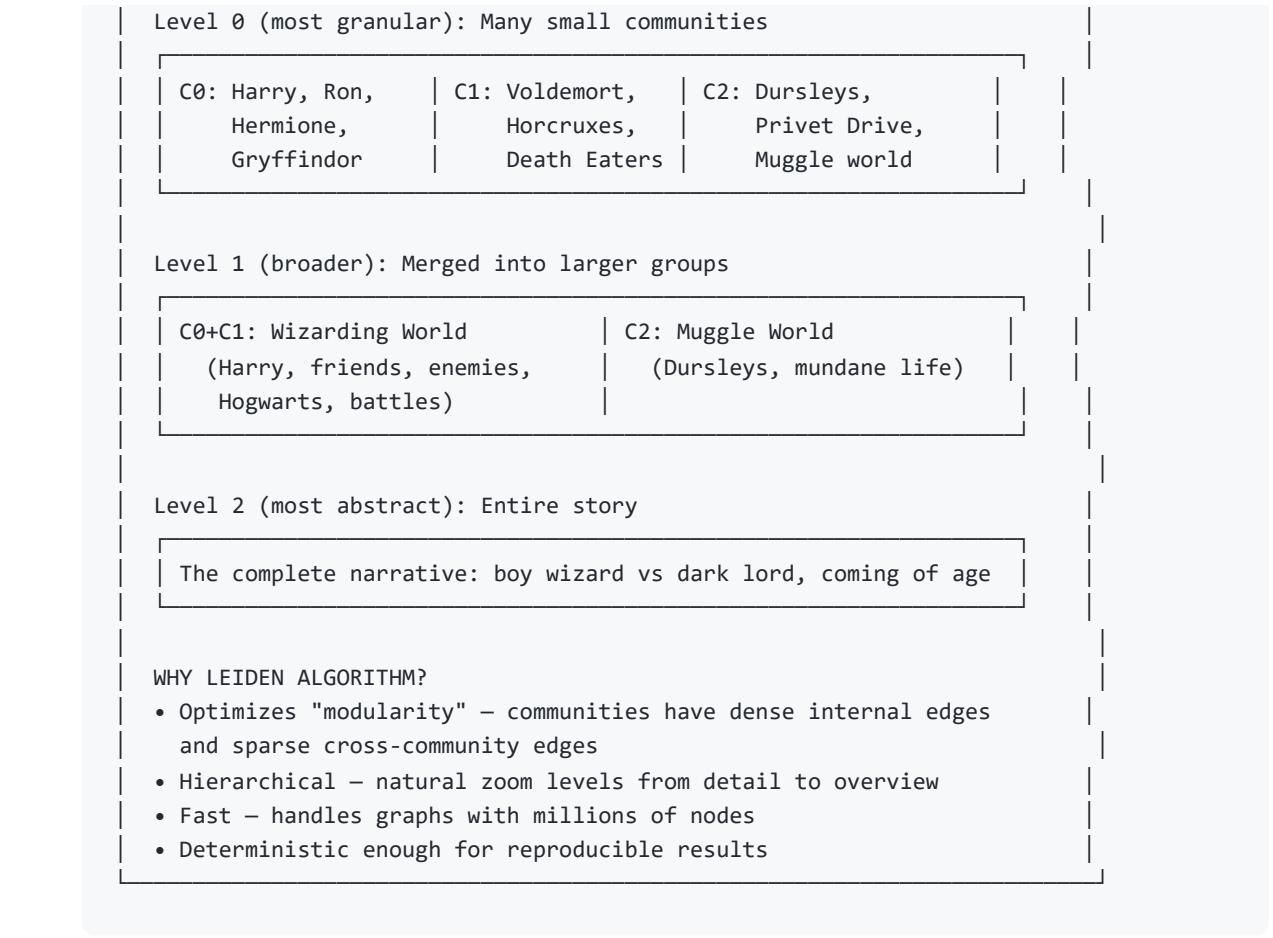



```
Nodes: ~50-500 entities per book chapter
Edges: ~100-2000 relationships per book chapter
```

Step 4: Community Detection (Leiden Algorithm)

This is the **key innovation** of GraphRAG. The Leiden algorithm partitions the graph into **communities** — clusters of densely connected entities. It works hierarchically, creating communities at multiple zoom levels.

```
┌──────────────────────────────────────────────────────────────────────────┐
│                  HIERARCHICAL COMMUNITY DETECTION                        │
│                                                                          │
```



Step 5: Community Summarization

For each community at each level, an LLM generates a summary report:

```
Community Report for C0 (Level 0): "The Hogwarts Trio"
──────────────────────────────────────────────────────

ENTITIES: Harry Potter (protagonist), Ron Weasley (loyal friend),
          Hermione Granger (brilliant strategist)

SUMMARY: Harry, Ron, and Hermione form the core trio at Hogwarts.
         Harry carries the burden of being "The Chosen One."
         Ron provides loyalty and emotional grounding despite
         insecurities about living in Harry's shadow. Hermione's
         intelligence and preparation save them repeatedly.
         Their friendship is tested by jealousy (Ron), isolation
         (Harry), and the pressure of their quest.

KEY RELATIONSHIPS:
  • Harry ↔ Ron: Deep friendship tested by jealousy in Year 4
  • Harry ↔ Hermione: Trust and mutual respect, platonic
  • Ron ↔ Hermione: Romantic tension, frequent arguments

THEMES: Friendship, loyalty, coming of age
```

These community summaries are **pre-computed** during indexing. At query time, the system just looks up the right summaries — no expensive graph traversal needed.

Local vs Global Search

GraphRAG supports two query modes:

Local Search: "Tell me about entity X"

Starts from a specific entity, traverses its neighborhood in the graph, and gathers relevant context.

```
Query: "What is Harry's relationship with Snape?"

LOCAL SEARCH PROCESS:
  1. Find entity "Harry" and "Snape" in the graph
  2. Traverse edges between them and nearby nodes
  3. Gather: direct relationships, shared communities, related entities
  4. Build context from entity descriptions + relationship descriptions
  5. LLM generates answer from this focused context

                    ┌──────────────┐
      teaches       │    Snape     │    protects (secretly)
    ◄───────────────│  (Potions    │──────────────────────┐
    │               │   Master)    │                      │
    │               └──────┬───────┘                      │
    │                      │ hates (surface)              │
    ▼                      ▼                              ▼
┌────────┐         ┌────────────┐                 ┌────────────┐
│Students│         │   Harry    │                 │   Lily     │
│        │         │            │                 │ (Harry's   │
└────────┘         └────────────┘                 │  mother)   │
                                                  └────────────┘
  Context gathered from these nodes + edges → LLM answer
```

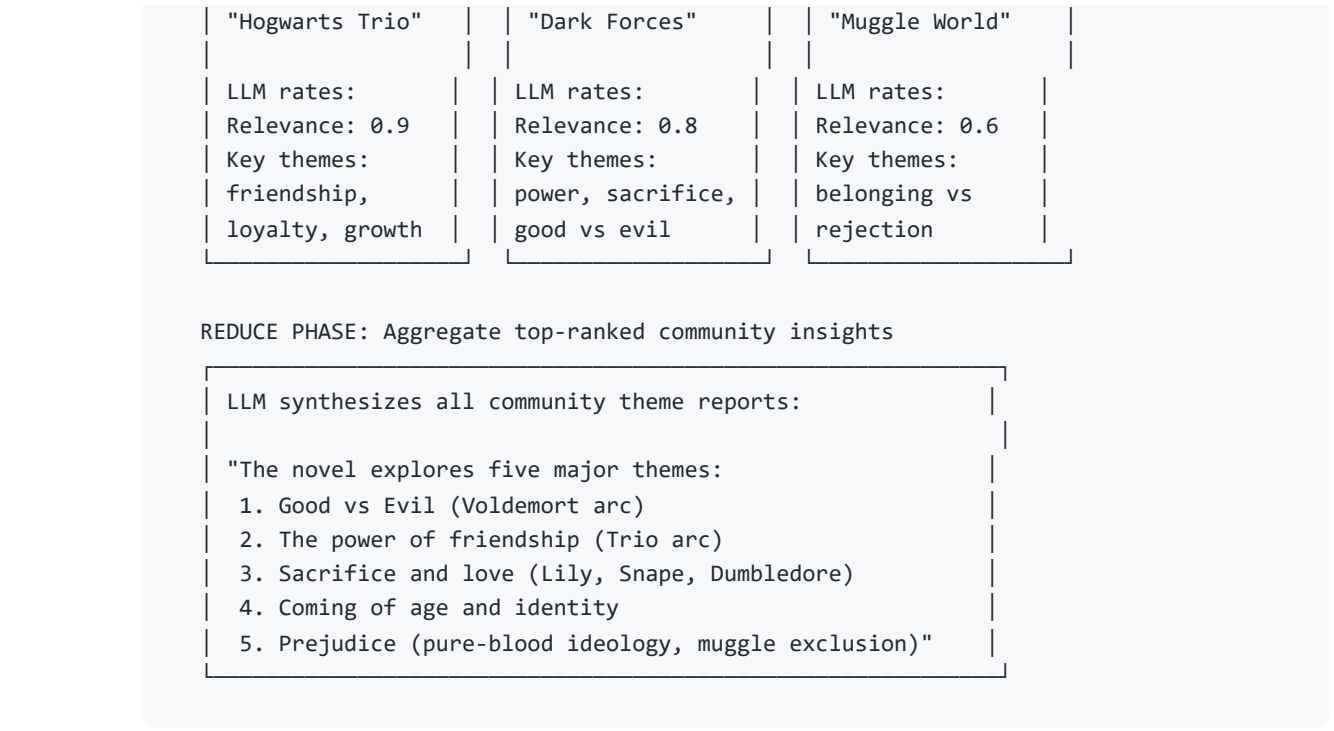
Global Search: "What are the main themes?"

Uses **map-reduce** over community summaries. This is what makes GraphRAG unique — it can answer questions about the ENTIRE corpus.

```
Query: "What are the major themes across this novel?"

GLOBAL SEARCH PROCESS (Map-Reduce):

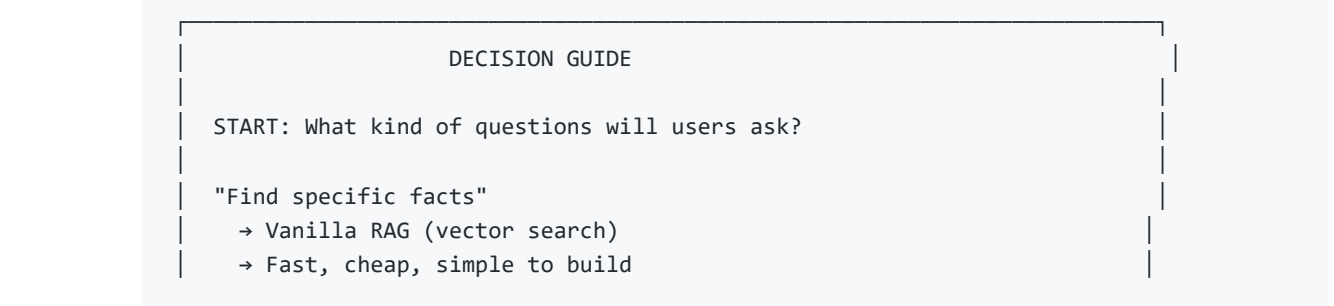
  MAP PHASE: Score each community summary against the query
  ┌──────────────────────┐  ┌──────────────────────┐  ┌──────────────────────┐
  │ Community 0          │  │ Community 1          │  │ Community 2          │
```



When to Use Each

Query Type	Search Mode	Example
Specific entity/fact	**Local**	"What happened to character X?"
Relationship between entities	**Local**	"How are A and B connected?"
Thematic / holistic	**Global**	"What are the main themes?"
Summarization	**Global**	"Summarize this 800-page book"
Comparison	**Global**	"Compare the leadership styles in this corpus"

When to Use GraphRAG vs Vanilla RAG



```
│                                                                          │
│  "Summarize / synthesize across many documents"                          │
│    → GraphRAG (community summaries)                                     │
│    → Expensive to build, but answers global questions                    │
│                                                                          │
│  "Both specific facts AND holistic questions"                            │
│    → Hybrid: GraphRAG for indexing + vector search for local queries     │
│                                                                          │
│  "Relationship-heavy questions (who knows whom, what caused what)"       │
│    → GraphRAG or dedicated Knowledge Graph (Neo4j)                       │
│    → Graph traversal excels at multi-hop reasoning                       │
└──────────────────────────────────────────────────────────────────────────┘
```

Dimension	Vanilla RAG	GraphRAG
Indexing cost	Low (embed chunks)	High (LLM extracts entities per chunk)
Indexing speed	Seconds-minutes	Minutes-hours
Query cost	Low (embed + top-k)	Medium (map-reduce over summaries)
Local questions	✅ Excellent	✅ Good (local search mode)
Global questions	❌ Poor	✅ Excellent
Relationship queries	❌ Poor	✅ Excellent
Multi-hop reasoning	❌ Poor	✅ Good
Freshness (updates)	Easy (re-embed new docs)	Hard (re-extract, re-cluster)
Best for	FAQ, search, Q&A	Analysis, summarization, research

Quality Monitoring & Evaluation

GraphRAG systems can silently degrade — extraction misses entities, summaries drift, communities fragment poorly. **You must monitor quality** at every stage.

The Evaluation Stack

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    GRAPHRAG QUALITY MONITORING LAYERS                    │
```

```
┌──────────────────────────────────────────────────────────────────────┐
│                                                                      │
│  Layer 1: EXTRACTION QUALITY                                         │
│  ┌──────────────────────────────────────────────────────────────────┐ │
│  │ Are entities extracted correctly? Are relationships accurate?    │ │
│  │                                                                  │ │
│  │ Metrics:                                                         │ │
│  │  • Entity Precision: % of extracted entities that are real       │ │
│  │  • Entity Recall: % of real entities that were extracted         │ │
│  │  • Relation Accuracy: % of extracted edges that are correct      │ │
│  │                                                                  │ │
│  │ How to measure:                                                  │ │
│  │  • Sample 50 chunks → manually verify extracted entities         │ │
│  │  • Compare against a hand-labeled "gold standard" for 1-2 chapters│ │
│  │  • Flag: Recall < 70% means you're losing critical entities      │ │
│  └──────────────────────────────────────────────────────────────────┘ │
│                                                                      │
│  Layer 2: GRAPH QUALITY                                              │
│  ┌──────────────────────────────────────────────────────────────────┐ │
│  │ Is the graph well-connected? Are communities meaningful?         │ │
│  │                                                                  │ │
│  │ Metrics:                                                         │ │
│  │  • Graph density: edges / (nodes × (nodes-1))                    │ │
│  │  • Isolated nodes: entities with 0 edges (lost context)          │ │
│  │  • Community modularity: quality of Leiden partitioning          │ │
│  │  • Giant component ratio: % of nodes in largest connected piece  │ │
│  │                                                                  │ │
│  │ Red flags:                                                       │ │
│  │  • >20% isolated nodes → extraction missed relationships         │ │
│  │  • Modularity < 0.3 → communities are poorly defined             │ │
│  │  • Giant component < 50% → graph is fragmented                   │ │
│  └──────────────────────────────────────────────────────────────────┘ │
│                                                                      │
│  Layer 3: SUMMARY QUALITY                                            │
│  ┌──────────────────────────────────────────────────────────────────┐ │
│  │ Are community summaries faithful, complete, and non-hallucinated? │ │
│  │                                                                  │ │
│  │ Metrics:                                                         │ │
│  │  • Faithfulness: Does the summary only state things supported    │ │
│  │    by the community's entities/edges? (no hallucination)         │ │
│  │  • Completeness: Does the summary mention all key entities in    │ │
│  │    the community? (no omissions)                                 │ │
│  │  • Informativeness: Is the summary useful or too generic?        │ │
│  │                                                                  │ │
│  │ How to measure:                                                  │ │
│  │  • RAGAS faithfulness metric (NLI-based)                         │ │
│  │  • Entity coverage check: list entities in community,            │ │
│  │    verify each appears in the summary                            │ │
│  └──────────────────────────────────────────────────────────────────┘ │
│                                                                      │
│  Layer 4: END-TO-END ANSWER QUALITY                                  │
│  ┌──────────────────────────────────────────────────────────────────┐ │
│  │ Does the final answer actually address the user's question?      │ │
│  │                                                                  │ │
│  │ Metrics (RAGAS framework):                                       │ │
│  │  • Faithfulness: answer grounded in retrieved context (0-1)      │ │
│  │  • Context Recall: relevant info retrieved (0-1)                 │ │
```

```
│  │  • Context Entities Recall: all relevant entities present (0-1)   │ │
│  │  • Answer Correctness: matches ground truth (0-1)                 │ │
│  │  • Answer Relevance: actually addresses the question (0-1)        │ │
│  └──────────────────────────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────────────────────┘
```

RAGAS: The Standard Evaluation Framework

RAGAS (Retrieval-Augmented Generation Assessment) is the most widely adopted framework for evaluating RAG systems, including GraphRAG. It works **without human-labeled ground truth** — using LLM-as-judge.

```
┌──────────────────────────────────────────────────────────────────────┐
│                    RAGAS METRICS EXPLAINED                           │
│                                                                      │
│  FAITHFULNESS (most critical for GraphRAG)                           │
│  ─────────────────────────────────────────                           │
│  "Does the answer ONLY say things supported by the context?"         │
│                                                                      │
│  How it works:                                                       │
│  1. LLM breaks the answer into atomic statements                     │
│  2. For each statement, check: "Is this supported by the context?"   │
│  3. Score = supported statements / total statements                  │
│                                                                      │
│  Answer: "Harry defeated Voldemort using the Elder Wand"             │
│  Statement 1: "Harry defeated Voldemort" → supported by context ✓    │
│  Statement 2: "He used the Elder Wand"   → supported by context ✓    │
│  Faithfulness = 2/2 = 1.0                                            │
│                                                                      │
│  CONTEXT RECALL                                                      │
│  ──────────────                                                      │
│  "Did we retrieve ALL the information needed to answer?"             │
│                                                                      │
│  Compares retrieved context against reference answer.                │
│  Low recall = important facts were never retrieved.                  │
│                                                                      │
│  CONTEXT ENTITIES RECALL (GraphRAG-specific)                         │
│  ──────────────────────────────────────────                          │
│  "Are all relevant ENTITIES present in the retrieved context?"       │
│                                                                      │
│  Expected entities: [Harry, Voldemort, Elder Wand, Dumbledore]       │
│  Found in context: [Harry, Voldemort, Elder Wand]                    │
│  Entity Recall = 3/4 = 0.75  (Dumbledore missing → context gap)      │
└──────────────────────────────────────────────────────────────────────┘
```

Practical Monitoring Pipeline

```
# Run after every indexing job AND periodically on live queries

MONITORING_CHECKS = {
```

```
    # Extraction health
    "entity_count": "expect 50-500 per chapter (too few = weak extraction)",
    "relationship_count": "expect 2-5x entity count",
    "isolated_nodes": "flag if > 20% of entities have zero edges",

    # Graph health
    "community_count": "expect 5-50 communities for a novel",
    "avg_community_size": "sweet spot: 5-30 entities per community",
    "modularity_score": "should be > 0.3 (Leiden quality metric)",

    # Summary health (sample 10 community summaries)
    "faithfulness_score": "RAGAS: should be > 0.8",
    "entity_coverage": "each community summary should mention 80%+ of its entities",

    # End-to-end (run a test suite of 20-30 questions)
    "answer_faithfulness": "should be > 0.8 (no hallucination)",
    "context_recall": "should be > 0.7 (retrieving enough info)",
    "entity_recall": "should be > 0.75 (not missing key entities)",
}
```

Alerting Thresholds

Metric	Green ✅	Yellow ⚠️	Red 🔴
Entity extraction recall	>80%	60-80%	<60%
Isolated nodes	<10%	10-20%	>20%
Community modularity	>0.4	0.3-0.4	<0.3
Faithfulness (RAGAS)	>0.85	0.7-0.85	<0.7
Context entities recall	>0.80	0.65-0.80	<0.65
Answer correctness	>0.80	0.6-0.80	<0.6

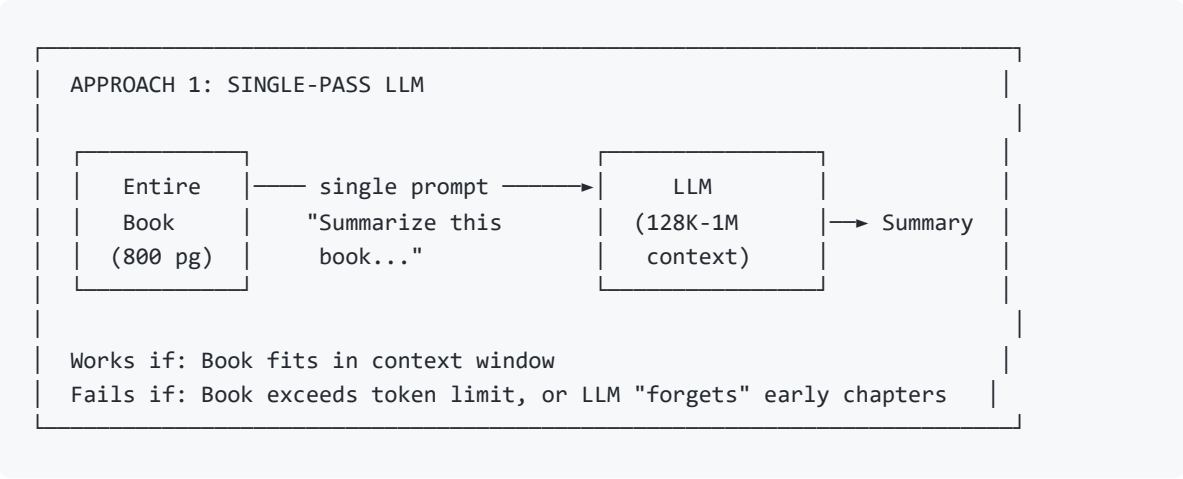
Practical Solution: Iterative Book Summarization

Now let's solve a real problem: **summarize an 800-page book into a good short story without missing characters, meaningful events, or narrative hints.**

Before jumping to the solution, let's evaluate all viable approaches honestly — each has real trade-offs.

Approach 1: Single-Pass LLM ("Just Send the Whole Book")

Feed the entire book into one LLM call and ask for a summary.



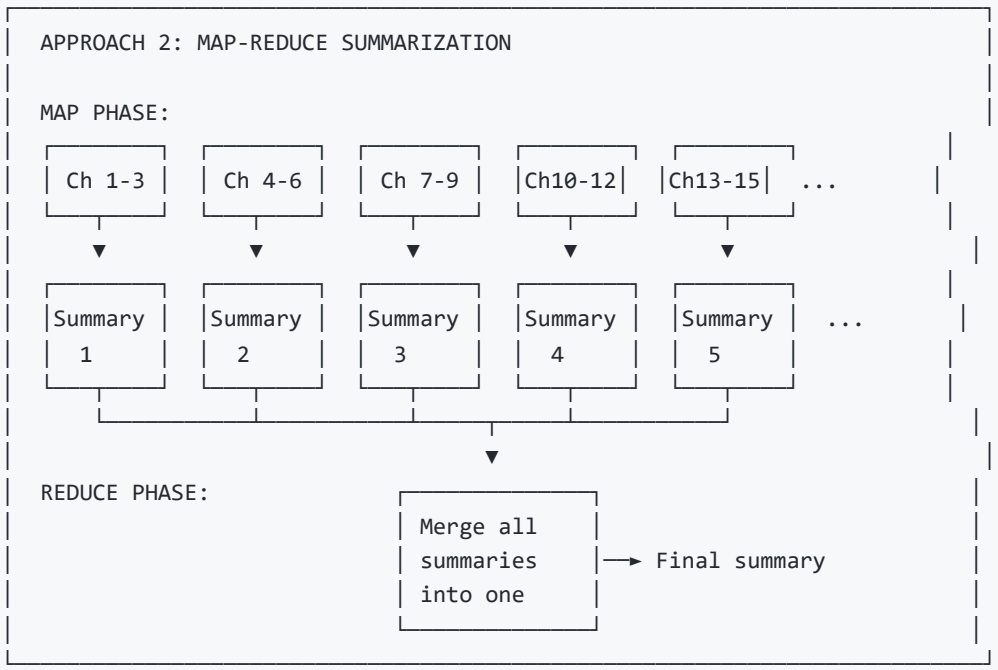
How it works: Load the full book text into the prompt and ask for a summary. With models like Gemini (1M tokens) or Claude (200K tokens), many books fit in a single pass.

Dimension	Assessment
✅ Simplicity	One prompt, one call, done
✅ Coherence	LLM sees everything at once — naturally coherent output
❌ Token limit	800 pages ≈ 400K tokens. Exceeds most models. Even if it fits, quality degrades at the edges
❌ "Lost in the middle"	LLMs attend strongly to the beginning and end of context, but lose detail in the middle. Characters introduced in chapter 15-30 (the "middle") get systematically dropped
❌ Cost	Sending 400K tokens at $15/M input = $6 per summary attempt. Iteration is expensive
❌ No verification	You get one shot — no way to check what was missed without reading the whole book yourself

Verdict: Works for short books (<200 pages). For long novels, the "lost in the middle" problem means you'll miss characters and subplots in the middle of the book. No mechanism to detect or fix gaps.

Approach 2: Chunk-and-Summarize (Map-Reduce)

Split the book into chunks, summarize each chunk independently, then summarize the summaries.



How it works: Break the book into segments (e.g., 3 chapters each). Summarize each segment independently. Then feed all segment summaries into a final "reduce" step that merges them.

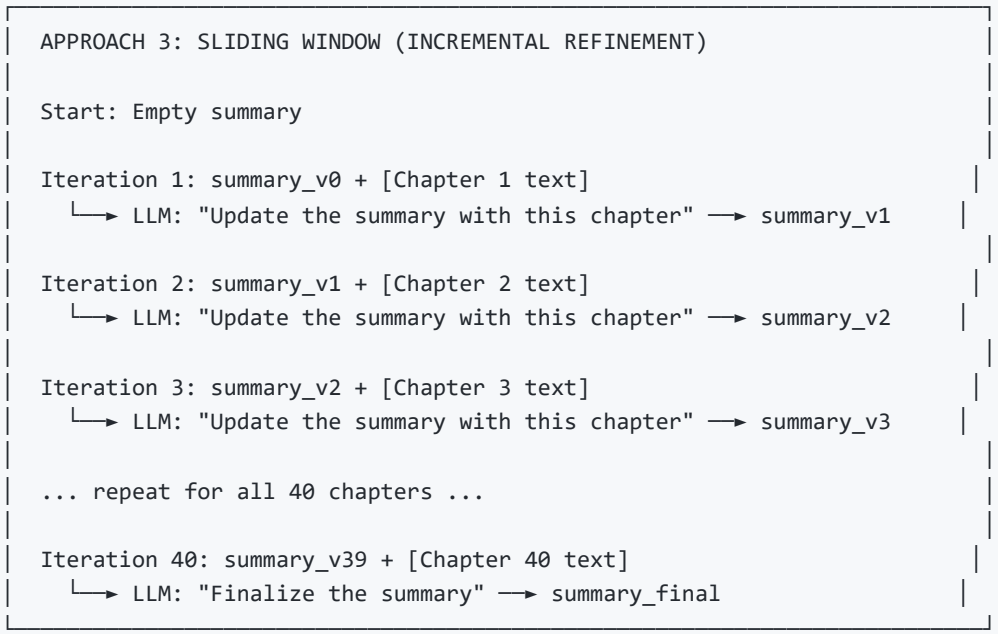
Dimension	Assessment
✅ Handles any length	Works on 100-page and 10,000-page books equally
✅ Parallelizable	Map phase runs all chunks in parallel — fast
✅ Simple to implement	Straightforward loop + merge
❌ Information loss	Each chunk is summarized in isolation. A character mentioned in chunk 3 and chunk 40 appears as two unrelated mentions
❌ No cross-chunk reasoning	Foreshadowing in chapter 5 has no connection to its payoff in chapter 40 — they're in different chunks
❌ Compression artifacts	"Summarize a summary" progressively loses detail. By the reduce step, minor-but-important characters are gone
❌ Reads like a list, not a story	The reduce step stitches together disconnected summaries. Result feels like "Chapter 1 is about X. Chapter 2 is about Y." Not a coherent narrative

Verdict: Good for factual reports ("What happened in each chapter?"). Poor for narrative summarization — loses character continuity, foreshadowing, and story arc because no chunk

sees the whole picture.

Approach 3: Sliding Window with Running Context

Maintain a "running summary" that gets updated as you process each chapter.



How it works: Process the book sequentially, chapter by chapter. After each chapter, ask the LLM to update a running summary that incorporates the new content. The summary grows and evolves as it reads.

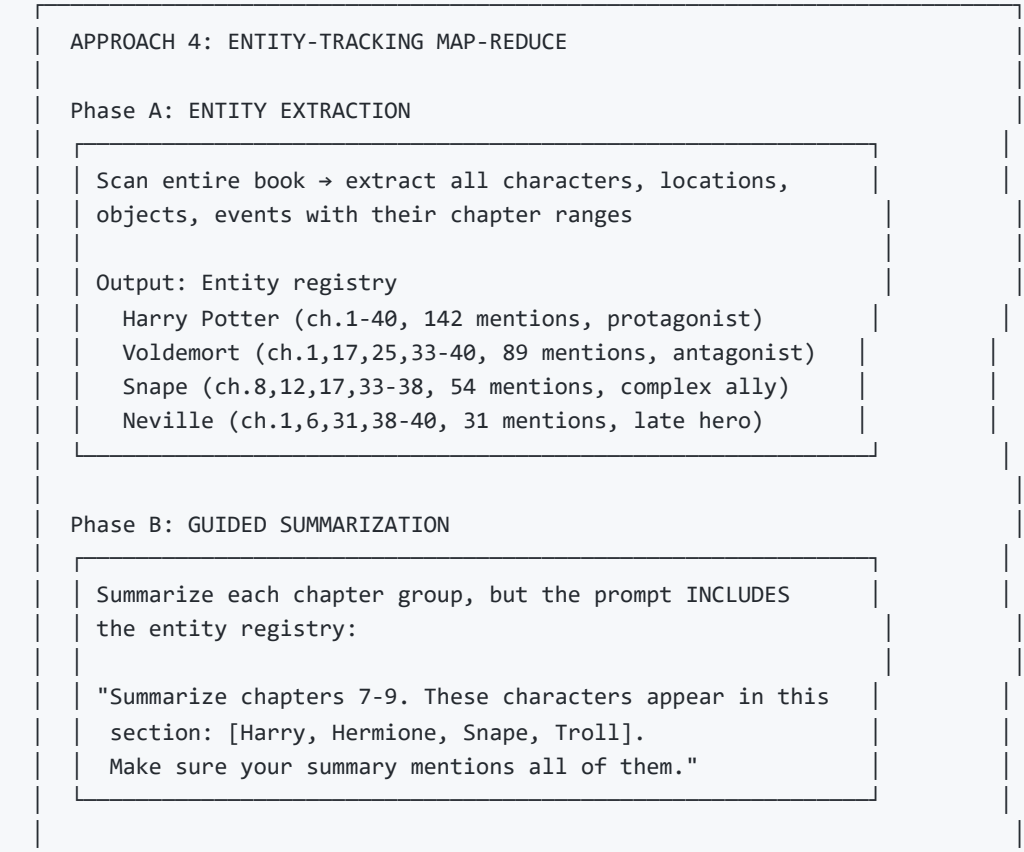
Dimension	Assessment
✅ Maintains context	The running summary carries forward character introductions and plot points
✅ Handles any length	Processes one chapter at a time — never exceeds context window
✅ Chronological narrative	Naturally builds a story in reading order
❌ Forward-bias	The summary is always written from the perspective of "what I've read so far." Early chapters get disproportionate detail because they were summarized when the summary was short. Later chapters get compressed

❌ Drift	After 40 iterations, the summary has drifted — early events get subtly reworded each iteration, losing precision
❌ No foreshadowing awareness	When processing chapter 5, the LLM doesn't know that the diary will matter in chapter 35. It may omit the diary as "minor detail"
❌ Slow	40 sequential LLM calls, each with the full running summary + chapter. Cannot parallelize
❌ No verification	Like Approach 1, you get one shot with no way to check completeness

Verdict: Better than map-reduce for narrative coherence. But suffers from forward-bias (can't weight early events by their future importance) and drift (40 rounds of summarize-the-summary degrades fidelity).

Approach 4: Entity-Tracking Map-Reduce

A smarter version of Approach 2: extract a character/entity list first, then use it to guide summarization.



```
│  Phase C: MERGE (same as Approach 2, but entity-aware)                  │
└─────────────────────────────────────────────────────────────────────────┘
```

Dimension	Assessment
✅ Character preservation	Entity registry ensures no important character is dropped
✅ Handles any length	Same chunked approach as map-reduce
✅ Better than naive map-reduce	Chunk summaries mention all relevant entities
❌ Still no cross-chunk reasoning	Knowing that Snape exists doesn't tell the summarizer HOW Snape connects to other characters across chapters
❌ Relationships are invisible	The entity list is flat — "Snape" and "Harry" appear as separate entries, not as "Snape protects Harry because he loved Lily"
❌ Foreshadowing still lost	Entities are tracked, but narrative seeds (the diary, the scar, the prophecy) aren't tracked as connected pairs

Verdict: A real improvement over naive map-reduce — characters won't vanish. But still misses the structural relationships that make a summary read like a story rather than a character inventory.

Approach 5: GraphRAG + Iterative Refinement ✅ (Recommended)

Build a knowledge graph of entities and relationships, use community detection to find narrative arcs, then iteratively summarize and validate.

```
┌─────────────────────────────────────────────────────────────────────────┐
│  APPROACH 5: GRAPHRAG + ITERATIVE VALIDATION                            │
│                                                                         │
│  Phase 1: Build Knowledge Graph (entities + RELATIONSHIPS)              │
│  Phase 2: Community detection (find narrative arcs)                     │
│  Phase 3: Hierarchical summarization (bottom-up)                        │
│  Phase 4: Automated validation (check for gaps)                         │
│  Phase 5: Targeted refinement (fix gaps, repeat)                        │
│                                                                         │
│  KEY ADVANTAGE: Relationships are first-class citizens.                 │
│  "Snape protects Harry because he loved Lily" is an EDGE in the graph,  │
│  not something that has to be inferred from flat text.                  │
│                                                                         │
│  KEY ADVANTAGE: Community detection finds narrative arcs automatically. │
│  The Leiden algorithm clusters densely-connected entities together —    │
│  these clusters naturally correspond to subplots and character arcs.    │
│                                                                         │
```

```
┌──────────────────────────────────────────────────────────────────────┐
│  KEY ADVANTAGE: Validation catches gaps.                             │
│  After summarizing, we CHECK whether important entities made it into │
│  the summary. If not, we fix it. This is what all other approaches   │
│  lack — a feedback loop.                                             │
└──────────────────────────────────────────────────────────────────────┘
```

Dimension	Assessment
✅ Relationship-aware	Knows WHO connects to WHOM and HOW — summaries read as stories
✅ Finds narrative arcs	Community detection groups characters by storyline automatically
✅ Foreshadowing preserved	Long-range graph edges (entity in ch.5 → same entity in ch.35) are explicitly tracked
✅ Verification loop	Automated checks catch missing characters, broken event chains, lost hints
✅ Iterative improvement	Gaps detected → fixed → re-checked until quality thresholds are met
❌ Expensive to build	Entity extraction requires one LLM call per chunk — hundreds of calls for a long book
❌ Extraction errors compound	If the LLM misses a character during extraction, they're missing from the graph entirely
❌ Complex pipeline	5 phases vs 1 prompt — significantly more engineering

Verdict: The most robust approach for long, complex narratives. The knowledge graph preserves what other approaches lose (relationships, narrative structure), and the validation loop catches what would otherwise silently vanish. The trade-off is cost and complexity — justified for high-quality summarization.

Approaches Comparison Summary

```
┌──────────────────────────────────────────────────────────────────────┐
│  APPROACH COMPARISON MATRIX                                          │
│                                                                      │
│                  Coherence  Characters  Foreshadow  Verification  Cost │
│  ──────────────────────────────────────────────────────────────────  │
│  1. Single-pass    ★★★★      ★★          ★★          ✗           $$   │
│  2. Map-reduce     ★★        ★★          ★           ✗           $    │
│  3. Sliding window ★★★       ★★★         ★           ✗           $$   │
│  4. Entity-track   ★★★       ★★★★        ★★          ✗           $$   │
```

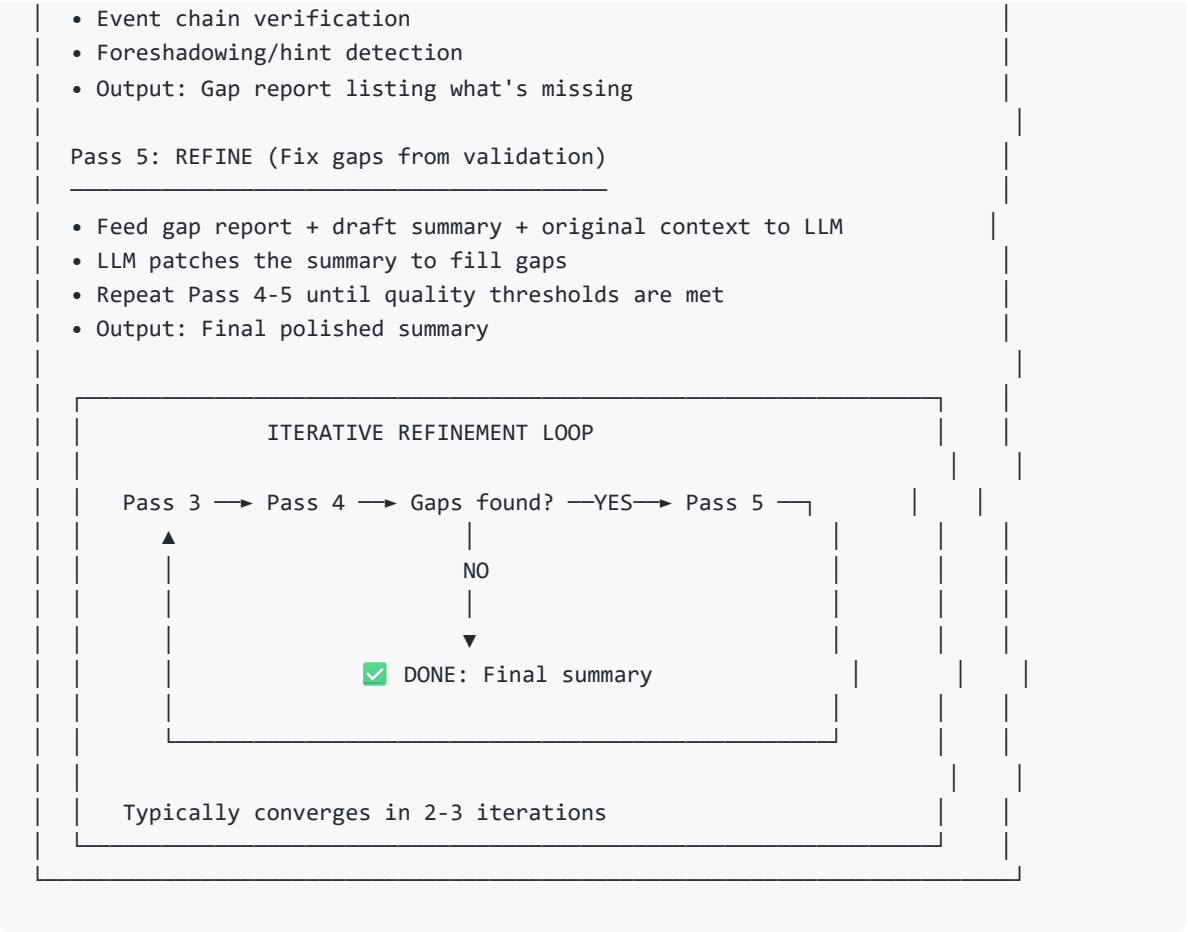
```
│  5. GraphRAG ✅      ★★★★★      ★★★★★       ★★★★★       ✅          $$$   │
│                                                                            │
│  ★ = poor  ★★★ = adequate  ★★★★★ = excellent  ✗ = none  ✅ = built-in      │
└────────────────────────────────────────────────────────────────────────────┘
```

Approach	Best For	Avoid When
1. Single-pass	Short books (<200 pages), quick-and-dirty	Long books, precision matters
2. Map-reduce	Factual summaries, reports	Narrative coherence matters
3. Sliding window	Medium books, acceptable quality	Foreshadowing or late-book callbacks matter
4. Entity-tracking	Good enough for most use cases	Complex relationship networks
5. GraphRAG	Long, complex novels with many characters and subplots	Simple documents, budget-constrained

---

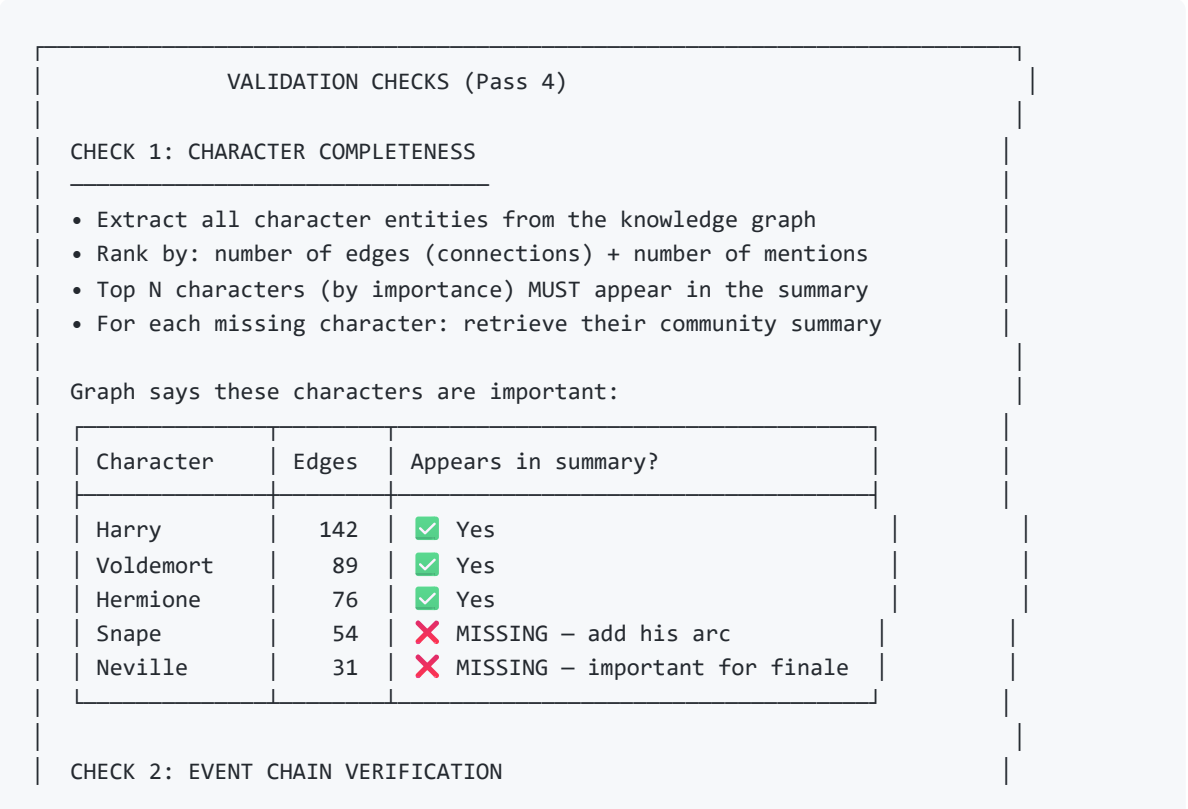
The Recommended Solution: GraphRAG 5-Pass Architecture

```
┌────────────────────────────────────────────────────────────────────────────┐
│              ITERATIVE BOOK SUMMARIZATION WITH GRAPHRAG                    │
│                                                                            │
│  Pass 1: EXTRACT (Build the knowledge graph)                               │
│  ──────────────────────────────────────────                                │
│  • Chunk the book (~600 tokens per chunk, 50-token overlap)                │
│  • LLM extracts entities & relationships from each chunk                   │
│  • Output: Knowledge graph with all characters, locations, events          │
│                                                                            │
│  Pass 2: STRUCTURE (Community detection + chapter mapping)                 │
│  ──────────────────────────────────────────────────────                    │
│  • Run Leiden algorithm → communities at 3 levels                          │
│  • Map communities to narrative arcs (which chapters they span)            │
│  • Output: Hierarchical community structure with temporal ordering         │
│                                                                            │
│  Pass 3: SUMMARIZE (Bottom-up community summaries)                         │
│  ──────────────────────────────────────────────                            │
│  • Level 0: Summarize each fine-grained community                         │
│  • Level 1: Merge into arc-level summaries                                 │
│  • Level 2: Synthesize into full story summary                             │
│  • Output: Draft summary (~2000-5000 words)                                │
│                                                                            │
│  Pass 4: VALIDATE (Automated quality checks)                               │
│  ──────────────────────────────────────────                                │
│  • Character completeness check                                            │
```



Pass 4 Detail: The Validation Checks

This is the critical innovation — **automated quality gates** that catch what's missing:



```
│  ──────────────────────────────────                                  │
│  • Extract event entities from the graph, ordered by chapter         │
│  • Check: does the summary preserve cause → effect chains?           │
│  • Missing causal link = gap in the narrative logic                  │
│                                                                      │
│  Event chain:                                                        │
│  Ch.1: Harry lives with Dursleys (setup)                             │
│  Ch.4: Harry gets Hogwarts letter (inciting incident)                │
│  Ch.17: Harry faces Voldemort (climax)                               │
│                                                                      │
│  Summary mentions Ch.4 and Ch.17 but not Ch.1 → ❌ GAP               │
│  Reader won't understand WHY the letter matters without the setup    │
│                                                                      │
│  CHECK 3: FORESHADOWING & HINTS                                      │
│  ──────────────────────────────                                      │
│  • Find relationships where entity A in chapter X connects to        │
│    entity B in chapter Y, where Y >> X (large chapter gap)           │
│  • These are narrative seeds — foreshadowing, Chekhov's guns         │
│  • The summary should either include the hint or the payoff          │
│                                                                      │
│  Graph edge: (diary, Ch.2) ──appears again──► (Horcrux reveal, Ch.35) │
│  Chapter gap: 33 chapters apart → likely foreshadowing               │
│  Summary mentions Horcrux reveal but not the diary → ❌ Missing hint  │
│                                                                      │
│  CHECK 4: NARRATIVE COHERENCE                                        │
│  ────────────────────────────                                        │
│  • Feed the summary to an LLM-as-judge with the prompt:              │
│    "Rate this summary on: Does it read as a coherent story?          │
│     Does it have a clear beginning, middle, and end?                 │
│     Would a reader who hasn't read the book understand the plot?"    │
│  • Score 1-5. If < 3, flag for rewrite.                              │
└──────────────────────────────────────────────────────────────────────┘
```

Pass 5 Detail: Targeted Refinement

```
Gap report from Pass 4:
  MISSING CHARACTERS: [Snape, Neville]
  MISSING EVENT LINKS: [Ch.1 setup → Ch.4 letter]
  MISSING HINTS: [diary → Horcrux connection]
  COHERENCE SCORE: 3.2/5 (needs improvement)

Pass 5 prompt to LLM:
  "Here is a draft summary of the book:
   [draft summary]

   The following gaps were detected:
   1. Snape is a critical character (54 graph edges) but missing.
      His community summary: [Snape's community report]
   2. Neville is important for the finale (31 graph edges) but missing.
      His community summary: [Neville's community report]
   3. The setup (Ch.1) explaining Harry's miserable life with
      the Dursleys is missing. This context is needed before
      the Hogwarts letter.
   4. The diary appears in Ch.2 and connects to the Horcrux
```

```
    reveal in Ch.35. Include this foreshadowing.

Please revise the summary to:
- Integrate these missing elements naturally (not as a list)
- Maintain narrative flow (beginning → middle → end)
- Keep the total length under 3000 words"
```

Tools & Frameworks

Tool	What It Does	When to Use
Microsoft GraphRAG	Full GraphRAG pipeline (extract, cluster, summarize, query)	Production-ready, end-to-end
LlamaIndex KnowledgeGraphIndex	Knowledge graph construction + query	When already using LlamaIndex
Neo4j + LangChain	Graph database + RAG integration	When you need a persistent, queryable graph DB
NetworkX	Python graph library (build, analyze, visualize)	Prototyping, small-medium graphs
graspologic	Leiden algorithm + graph statistics	Community detection (Microsoft's lib)
RAGAS	Evaluation framework for RAG systems	Quality monitoring and testing
graphrag-eval	GraphRAG-specific evaluation metrics	Benchmarking entity coverage

Microsoft GraphRAG Quick Start

```
# Install
pip install graphrag

# Initialize project
graphrag init --root ./my-book-project

# Place your book in ./my-book-project/input/book.txt

# Run indexing (builds the knowledge graph)
graphrag index --root ./my-book-project
```

```
# Query: Local search (entity-focused)
graphrag query --root ./my-book-project \
  --method local \
  --query "What is the relationship between Harry and Snape?"

# Query: Global search (holistic/thematic)
graphrag query --root ./my-book-project \
  --method global \
  --query "What are the main themes of this book?"
```

References

Resource	Type	Description
From Local to Global (arXiv:2404.16130)	Paper	The original Microsoft GraphRAG paper
microsoft/graphrag	Code	Official open-source implementation
Essential GraphRAG (Manning/Neo4j)	Book	Free book on GraphRAG with Neo4j
RAGAS Documentation	Docs	Evaluation framework for RAG/GraphRAG
GraphRAG: Dynamic Community Selection	Blog	Microsoft's improvements to global search
Leiden Algorithm	Paper	The community detection algorithm used

---

← Back to RAG + Vector Memory | Back to Syllabus